Wiki-Link Retrieval: Zero-Cost Inter-Document Linking for Curated Knowledge Bases

Anonymous Author(s)
Affiliation
Address
email

Abstract

Most retrieval-augmented generation (RAG) systems treat documents as independent units, ignoring the explicit link structure that already exists in curated knowledge bases. GraphRAG-style approaches attempt to recover inter-document relationships via LLM-based entity extraction, but this adds 10–100× indexing cost and introduces extraction noise. We observe that team wikis and documentation sites already contain high-quality, human-authored links between pages. We propose **Wiki-Link Retrieval**: parsing existing wiki link structure into an adjacency list and using it for post-retrieval 1-hop expansion after standard ANNS search. On a controlled 20-document corpus, link expansion improves Recall@5 by +3.5% overall and +8.3% on hard-link queries where the relevant document has low lexical overlap. Wiki-link retrieval achieves 2.2× better Recall@5 and 3.2× better MRR than a GraphRAG baseline with only ∼1 ms additional latency — no entity extraction, no graph database, no extra LLM calls.

1 Introduction

Retrieval-augmented generation (RAG) has become the standard approach for grounding large language models in domain-specific knowledge. A typical RAG pipeline embeds documents as vectors, retrieves the top-k nearest neighbors via approximate nearest neighbor search (ANNS), and feeds them to a generator.

However, this approach treats each document as an **isolated unit**. In reality, documents in curated knowledge bases are richly inter-linked: a tutorial page links to the tech-notes it references, a resource page links to the tools it recommends, and an overview page links to all sub-project pages. These links encode semantic relationships that are **invisible to vector similarity**.

GraphRAG and property graph approaches attempt to recover inter-document relationships by running LLM-based entity extraction over raw text. While powerful, this introduces significant costs:

- **Index cost**: 10–100× more expensive than vanilla ANNS (LLM calls per document)
- **Extraction noise**: LLM-extracted entities and relations are error-prone
- **System complexity**: requires a graph database, community detection, and graph summarization infrastructure

We observe that **curated wikis already contain the link graph for free**. Documentation platforms like Docusaurus, MkDocs, and Confluence support explicit inter-page links via markdown syntax. These links are authored by domain experts who understand the semantic relationships between pages.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: How mainstream RAG/wiki solutions handle inter-document linking

Approach	Examples	Inter-Linking Strategy	Index Cost	Query Co
Wiki + Vector Search	Notion AI, Dify, FastGPT	None at retrieval	Low: $O(n)$ embed	ANNS on
GraphRAG	Microsoft GraphRAG, LightRAG	Full KG: entities + relations	**High: 10–100**$\times$	ANNS +
Property Graph	LlamaIndex, Zep, Neo4j	(S,P,O) triples in graph DB	Medium: NER + triple	+50–200r
Hierarchical Memory	Mem0, MemGPT	Working$\rightarrow$episodic$\rightarrow$semantic	Medium: multi-level	Multi-leve
Wiki-Link (ours)	—	Parse wiki links $\rightarrow$ adjacency $\rightarrow$ 1-hop	**Low:** $O(n)$**+**$O(\|E\|)$	**+**$\sim$**1ms**

Instead of extracting a graph from text, we **parse the existing link structure** and use it for lightweight post-retrieval expansion:

Query $\rightarrow$ ANNS retrieves top-k documents
$\rightarrow$ 1-hop link expansion via pre-built adjacency list
$\rightarrow$ Score: primary results + decay-weighted linked context
$\rightarrow$ Return augmented top-k

Our contributions:

1. **Landscape analysis** showing that no mainstream RAG system exploits curated wiki links at retrieval time (§2)
2. **System design** for zero-cost link graph construction and post-retrieval expansion with bidirectional adjacency (§3)
3. **Auto-sync pipeline** that keeps the link graph current with wiki edits at negligible maintenance cost (§3.4)
4. **Empirical evaluation** demonstrating measurable recall improvement on hard-link queries and 2.2$\times$ advantage over GraphRAG (§4)

2 Related Work: The Inter-Linking Gap in RAG Systems

2.1 Landscape of Current Approaches

Table 1 surveys how mainstream LLM-powered knowledge systems handle inter-document linking. The key finding: **no existing system exploits curated wiki links at the retrieval level**.

2.2 Analysis of the Gap

Wiki + Vector Search systems (Notion AI, Confluence AI, Dify, FastGPT) represent the status quo. They index wiki pages as independent documents and retrieve by vector similarity. While their UIs often display "related pages" sidebars derived from wiki links, these links are **never used at the retrieval level**. A user querying "how to handle GPU OOM" will only get documents with high lexical/semantic overlap — even if a closely related page is one click away in the wiki.

GraphRAG [Research, 2024] addresses this gap by extracting entity graphs from raw text using LLMs. It builds community summaries at index time and uses them for global query answering. While powerful for exploratory queries, it has three drawbacks for curated knowledge bases: (1) indexing cost is 10–100$\times$ higher due to LLM calls, (2) extracted entities are noisy compared to human-authored links, and (3) the system requires a graph database and community detection infrastructure.

Property Graph approaches (LlamaIndex, Zep, Neo4j) extract structured (subject, predicate, object) triples and store them in a graph database. Queries use Cypher or GQL for precise relationship traversal. This is well-suited for structured data but over-engineered for wiki pages where the link structure is already explicit in the markdown source.

Hierarchical Memory systems (Mem0 [mem, 2024], MemGPT [Packer et al., 2024]) organize knowledge into working/episodic/semantic tiers. While they model temporal dynamics, they do not address inter-document linking — a page about KV Cache optimization and a page about NPU memory management are stored independently even if the wiki explicitly links them.

Algorithm 1 Link Graph Construction

```
function REBUILDLINKGRAPH(𝒟)
    for each document D ∈ 𝒟 do
        for each linked source S ∈ D.metadata[linked_source_names] do
            Add edge D → S                                   ▷ forward
            Add edge S → D                                   ▷ reverse
        end for
    end for
end function
```

Algorithm 2 Post-Retrieval 1-Hop Link Expansion

```
function EXPANDHITSWITHLINKS(ℛ, G, m, α)
    ℛ′ ← ℛ
    c ← 0
    for each hit H ∈ ℛ do
        for each neighbor N ∈ G[H.document_id] do
            if N ∉ ℛ′ then
                Add N to ℛ′ with score = H.score × α
                c ← c + 1
            end if
            if c ≥ m then
                return ℛ′
            end if
        end for
    end for
    return ℛ′
end function
```

2.3 Our Position

Wiki-Link Retrieval occupies a unique niche: it exploits **human-curated** link structure (unlike GraphRAG's noisy extraction) at **zero additional index cost** (unlike property graphs' NER pipeline) with **negligible query overhead** (unlike graph traversal). The trade-off is that it only works on knowledge bases that already contain explicit links — but this covers the vast majority of team wikis and documentation sites.

3 Method

3.1 Link Graph Construction

At ingestion time, we parse wiki markdown files for inter-page links (e.g., `[text](../other-page.md)`). Each link is resolved to a canonical source name (e.g., `wiki:tech-notes/kv-cache-optimization`) and stored as document metadata.

At index build time, we construct a **bidirectional adjacency list**: if document A links to document B, both $A \to B$ and $B \to A$ are added as valid expansion edges. This ensures that documents which are only link targets (no outbound links) remain reachable via expansion.

Graph construction is $O(|E|)$ where E is the total number of links — typically a few hundred for a team wiki. No LLM calls, no entity extraction, no graph database.

3.2 Post-Retrieval 1-Hop Expansion

At query time, after ANNS retrieval and reranking produce the initial top-k results, we perform 1-hop link expansion:

Key parameters:

- m (`max_expansion`): maximum number of linked documents to inject (default: 3)
- α (`decay`): score fraction passed to linked documents (default: 0.5)

3.3 Optimal Decay Factor

Our ablation study (§4.4) reveals a counter-intuitive finding: **lower decay ($\alpha = 0.3$) outperforms higher decay ($\alpha \in [0.5, 1.0]$)**. With $\alpha = 0.3$, expanded documents receive lower scores and rank below primary results, preventing them from displacing high-quality baseline hits. With $\alpha = 1.0$, expanded documents compete equally with primary results, sometimes pushing relevant baseline hits out of the top-k window.

Recommendation: Use $\alpha = 0.3$ for conservative expansion that augments without disrupting.

3.4 Automatic Knowledge Synchronization

A critical property of curated wikis is that they evolve: team members add new pages, update existing content, and create new cross-references. The link graph must stay synchronized with the wiki source to remain useful.

We implement a **zero-maintenance auto-sync pipeline**:

$$\texttt{wiki push (git)} \rightarrow \texttt{systemd timer (every 30 min)} \rightarrow \texttt{git pull} \rightarrow \texttt{ingest\_wiki.py (idempotent upsert)} \rightarrow \texttt{rebuild link graph} \rightarrow \texttt{KB live}$$

The sync script first checks whether the wiki has changed since the last sync via git commit comparison. If unchanged, ingestion is skipped entirely — making the overhead of no-op syncs negligible. When changes are detected, all documents are re-ingested via upsert (create new, update existing), and the link graph is fully rebuilt.

This contrasts sharply with GraphRAG, where wiki updates require re-running the expensive entity extraction pipeline over all documents.

4 Experiments

4.1 Experimental Setup

Corpus: 20 interconnected wiki documents with deliberate hub/spoke structure. Hub documents (tutorials, overviews) have high keyword overlap with queries and many outbound links. Spoke documents (deep tech-notes) have specific technical content and lower keyword overlap.

Ground truth: 20 queries across three difficulty levels:

- **Easy** (5): direct keyword match
- **Medium** (5): multi-document, partial expansion benefit
- **Hard-link** (10): relevant document has low keyword overlap, reachable only via links from hub documents

Graph statistics: 20 nodes, 88 bidirectional edges, average degree 4.4.

Baselines:

- Vanilla ANNS (token-overlap search, no expansion)
- GraphRAG (entity co-occurrence graph with 1-hop entity expansion)

4.2 Main Results

Table 2 summarizes the main results. Key findings:

- Link expansion improves Recall@5 by **+3.5%** ($\alpha = 0.5$) or **+7.0%** ($\alpha = 0.3$) over baseline

Table 2: Wiki-Link Retrieval vs baselines on 20-document corpus ($k = 5$)

Method	Recall@5	MRR	Latency
Vanilla ANNS (baseline)	0.717	0.887	1.5ms
Wiki-Link ($\alpha = 0.5$)	**0.742**	0.887	1.4ms
Wiki-Link ($\alpha = 0.3$)	**0.767**	0.887	1.4ms
GraphRAG (1-hop entity)	0.342	0.277	0.1ms

- Wiki-link achieves **2.2× better Recall@5** and **3.2× better MRR** than GraphRAG on the same corpus
- Latency overhead is **negligible** ($\sim$1ms for dict lookups)

4.3 Hard-Link Query Analysis

On the 10 hard-link queries (where relevant docs have low keyword overlap):

Table 3: Hard-link query performance ($n = 10$)

Metric	Baseline	Wiki-Link ($\alpha = 0.5$)
Recall@5	0.600	0.650
Δ	—	**+0.050**

The largest single-query improvement: a query about "how to apply for lab GPU access" went from Recall@5=0.50 to 1.00 (+0.50), because link expansion found the NPU setup tutorial via the cluster guide page.

4.4 Ablation Studies

4.5 GraphRAG Comparison

The GraphRAG baseline uses entity co-occurrence from regex NER (50 entities, 328 edges). Despite having 3.7× more edges than the wiki-link graph, it achieves significantly worse retrieval quality because entity co-occurrence is a noisy signal compared to human-authored links.

5 Discussion

5.1 When Does Link Expansion Help Most?

Link expansion is most valuable when:

1. **The relevant document has low lexical overlap** with the query but is linked from a keyword-rich hub document
2. **The wiki has hub/spoke structure** with overview pages linking to detailed technical pages
3. **The query is "hard"** — requiring context that goes beyond direct keyword matching

5.2 Limitations

1. **Requires existing link structure**: only works on wikis/docs with explicit inter-page links. Unlinked or poorly linked wikis won't benefit.
2. **Expansion budget problem**: in dense graphs, most neighbors of a hit are already in the baseline result set, wasting the expansion budget.
3. **Small corpus evaluation**: our controlled experiment uses 20 documents. Scaling to larger, mixed KBs (hundreds of non-wiki documents) requires further investigation — on a 635-document KB with only 18 wiki pages, expansion showed $\Delta = 0$ because wiki pages were already found by baseline.

Table 4: Ablation on decay factor α ($k = 5$, $m = 3$, bidirectional)

α	**Recall@5**	**MRR**	**Interpretation**
0.3	**0.767**	0.887	Best: expanded docs rank low
0.5	0.742	0.887	Good: balanced
0.7	0.742	0.875	Expanded docs compete
1.0	0.717	0.863	Worst: equals baseline

Table 5: Ablation on max expansion m ($k = 5$, $\alpha = 0.5$, bidirectional)

m	**Recall@5**	**Note**
1	0.742	Same as higher values
3	0.742	Budget consumed by seen neighbors
5	0.742	No additional benefit
10	0.742	Diminishing returns

5.3 Future Work

- **Smart expansion**: skip neighbors already in baseline without counting against the expansion budget
- **2-hop expansion**: neighbors-of-neighbors with deeper decay ($\alpha^2 = 0.09$)
- **Hybrid KBs**: combining wiki-link expansion with broader document retrieval in mixed-corpora settings
- **LLM-based GraphRAG comparison**: replace regex NER with actual LLM extraction for a fairer comparison

6 Conclusion

We presented Wiki-Link Retrieval, a lightweight approach that exploits human-authored link structure in curated wikis for post-retrieval expansion. With zero additional indexing cost (no LLM calls, no graph database) and negligible query overhead ($\sim$1ms), it improves Recall@5 by up to +7% and outperforms GraphRAG by 2.2$\times$ on the same corpus. The approach fills a gap in the current RAG landscape where no mainstream system exploits wiki links at the retrieval level.

References

Mem0: The memory layer for personalized AI, 2024. URL `https://github.com/mem0ai/mem0`.

Charles Packer, Shishir Wooders, Kevin Lin, Hannah Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024.

Microsoft Research. From local to global: A graph RAG approach to query-focused summarization. 2024. URL `https://github.com/microsoft/graphrag`. Microsoft GraphRAG.

Table 6: Ablation on link direction ($k = 5$, $m = 3$, $\alpha = 0.5$)

Direction	**Recall@5**	**Note**
Outbound-only	0.742	Forward edges from hubs
Bidirectional	0.742	Same on this corpus
Baseline (no exp)	0.717	Reference

Table 7: System complexity comparison

Dimension	**GraphRAG**	**Wiki-Link**
Graph nodes	50 entities	20 documents
Graph edges	328	88
Entity extraction	LLM/NER required	None
Graph database	Required	Not needed
Index build cost	High (LLM calls)	Low (link parse)
Human curation	None	Wiki links (existing)